



Basi di Dati

Progetto A.A. 2025/2026

SISTEMA INFORMATIVO DI UNA PIZZERIA

0315010

Davide Tamburrini

Indice

1. Descrizione del Minimondo	1
2. Analisi dei Requisiti	2
3. Progettazione concettuale.....	8
4. Progettazione logica	13
5. Progettazione fisica	24

1. Descrizione del Minimondo

1 Si vuole realizzare un sistema di gestione di una pizzeria, operante 7 giorni su 7 sia a
2 pranzo che a cena. La pizzeria opera in loco, tramite asporto o con consegne. Quando
3 un cliente entra in pizzeria, se vuole consumare sul posto, gli viene assegnato un tavolo
4 libero. I camerieri prendono le comande andando al tavolo ed aggiungendo le
5 ordinazioni all'elenco dei prodotti da preparare. I prodotti, appartenenti ad un menu
6 fisso gestito dal proprietario del locale, sono pizze o bevande. I pizzaioli possono
7 consultare il sistema per sapere quali pizze devono essere preparate. Il barman può
8 fare lo stesso per preparare le bevande. Quando una pizza o una bevanda (di una certa
9 comanda) è pronta, il pizzaiolo o il barman la indicano come preparata. I camerieri
10 possono consultare dai loro terminali quali prodotti sono pronti e a quali tavoli vanno
11 consegnati. In caso di consegne, l'ordine viene acquisito tramite telefono. In questo
12 caso, il proprietario registra l'indirizzo di consegna e il nome del cliente, oltre ad un
13 recapito telefonico. In caso di ordine da asporto o in consegna, il manager ritira sia pizze
14 che bevande dalla cucina, per consegnarle al cliente o al fattorino. Al termine del pasto
15 o al ritiro dell'ordine, il gestore segna la comanda come completata e può stampare lo
16 scontrino non fiscale di riepilogo. Per motivi statistici, ha la possibilità di visualizzare
17 le entrate giornaliere e/o mensili.
18
19
20
21

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
2	In loco	Sul posto	Mantenimento di una costanza terminologica con "consumare sul posto"
4	ordinazioni	Prodotti richiesti	Il termine indicava i singoli prodotti di una comanda, non la comanda stessa; si esplicita per evitare ambiguità
6,11	proprietario	gestore	Il ruolo che definisce il menu, registra gli ordini telefonici e gestisce le comande appare essere unico in tutta la specifica; si uniforma sotto un unico termine
13	manager	gestore	Stesso ruolo indicato altrove come "proprietario"/"gestore"; mantenimento di una costanza terminologica
11, 13, 15	ordine / l'ordine / dell'ordine	comanda / la comanda / della comanda	Il termine indica lo stesso concetto già espresso come "comanda"

Specifica disambiguata

Si vuole realizzare un sistema di gestione di una pizzeria, operante 7 giorni su 7 sia a pranzo che a cena. La pizzeria opera sul posto, tramite asporto o con consegne. Quando un cliente entra in pizzeria, se vuole consumare sul posto, gli viene assegnato un tavolo libero. I camerieri prendono le comande andando al tavolo ed aggiungendo i prodotti richiesti all'elenco di quelli da preparare. I prodotti, appartenenti ad un menu fisso definito dal gestore del locale, sono pizze o bevande. I pizzaioli possono consultare il sistema per sapere quali pizze devono essere preparate. Il barman può fare lo stesso per preparare le bevande. Quando una pizza o una bevanda (di una certa comanda) è pronta, il pizzaiolo o il barman la indicano come preparata. I camerieri possono consultare dai loro terminali quali prodotti sono pronti e a quali tavoli vanno consegnati. In caso di consegne, la comanda viene acquisita tramite telefono. In questo caso, il gestore registra l'indirizzo di consegna e il nome del cliente, oltre ad un recapito telefonico. In caso di comanda da asporto o in consegna, il gestore ritira sia pizze che bevande dalla cucina, per consegnarle al cliente o al fattorino. Al termine del pasto o al ritiro della comanda, il gestore segna la comanda come completata e può stampare lo scontrino non fiscale di riepilogo. Per motivi statistici, ha la possibilità di visualizzare le entrate giornaliere e/o mensili.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
---------	-------------	----------	--------------

Cliente	Colui che si reca in pizzeria per consumare sul posto, oppure che effettua una comanda da asporto o in consegna. Non ha accesso diretto al sistema.		Comanda, Tavolo, Gestore
Tavolo	Postazione fisica del locale assegnata a un cliente che desidera consumare sul posto, presso la quale viene aperta la comanda relativa.		Cliente, Comanda, Cameriere
Comanda	Insieme dei prodotti richiesti da un cliente in un'unica visita o ordine, sia essa consumata sul posto, ritirata come asporto o recapitata tramite consegna. Viene gestita dal gestore fino al suo completamento.	Ordine	Prodotto, Tavolo, Cliente, Gestore
Prodotto	Elemento del menu fisso del locale che può essere richiesto all'interno di una comanda. Si distingue in pizza o bevanda.		Pizza, Bevanda, Menu, Comanda
Pizza	Prodotto del menu, preparato dal pizzaiolo su richiesta di una comanda.		Prodotto, Pizzaiolo, Comanda
Bevanda	Prodotto del menu, preparato dal barman su richiesta di una comanda.		Prodotto, Barman, Comanda
Menu	Insieme fisso dei prodotti (pizze e bevande) offerti dalla pizzeria, definito dal gestore		Prodotto, Gestore
Cameriere	Lavoratore che assegna i tavoli ai clienti, prende le comande recandosi al tavolo e consulta quali prodotti sono pronti per la consegna.		Tavolo, Comanda, Prodotto

Pizzaiolo	Lavoratore che consulta il sistema per sapere quali pizze deve preparare e le segna come pronte una volta completate.		Pizza, Comanda
Barman	Lavoratore che consulta il sistema per sapere quali bevande deve preparare e le segna come pronte una volta completate.		Bevanda, Comanda
Gestore	Lavoratore che definisce il menu, registra le comande acquisite telefonicamente per le consegne, ritira i prodotti pronti per le comande da asporto o in consegna, segna le comande come completate, stampa lo scontrino non fiscale e consulta le entrate giornaliere e mensili.		Menu, Comanda, Prodotto, Cliente
Fattorino	Persona esterna al sistema a cui il gestore affida pizze e bevande pronte per una comanda in consegna, affinché le recapiti al cliente. Non interagisce direttamente con il sistema.		Comanda, Gestore

Raggruppamento dei requisiti in insiemi omogenei

Frase relative a Cliente
- Quando un cliente entra in pizzeria, se vuole consumare sul posto, gli viene assegnato un tavolo libero; - Il gestore registra l'indirizzo di consegna e il nome del cliente, oltre ad un recapito telefonico; - Il gestore ritira sia pizze che bevande dalla cucina, per consegnarle al cliente o al fattorino.

Frase relative a Tavolo

- Gli viene assegnato un tavolo libero;
- I camerieri prendono le comande andando al tavolo;
- I camerieri possono consultare dai loro terminali quali prodotti sono pronti e a quali tavoli vanno consegnati

Fraasi relative a Comanda

- I camerieri prendono le comande andando al tavolo ed aggiungendo i prodotti richiesti all'elenco di quelli da preparare;
- Quando una pizza o una bevanda (di una certa comanda) è pronta, il pizzaiolo o il barman la indicano come preparata;
- In caso di consegne, la comanda viene acquisita tramite telefono;
- In caso di comanda da asporto o in consegna, il gestore ritira sia pizze che bevande dalla cucina, per consegnarle al cliente o al fattorino;
- Al termine del pasto o al ritiro della comanda, il gestore segna la comanda come completata e può stampare lo scontrino non fiscale di riepilogo

Fraasi relative a Prodotto

- I camerieri prendono le comande andando al tavolo ed aggiungendo i prodotti richiesti all'elenco di quelli da preparare;
- I prodotti, appartenenti ad un menu fisso definito dal gestore del locale, sono pizze o bevande;
- I camerieri possono consultare dai loro terminali quali prodotti sono pronti e a quali tavoli vanno consegnati;
- Il gestore ritira sia pizze che bevande dalla cucina.

Fraasi relative a Pizza

- I prodotti... sono pizze o bevande;
- I pizzaioli possono consultare il sistema per sapere quali pizze devono essere preparate;
- Quando una pizza... è pronta, il pizzaiolo... la indica come preparata;
- Il gestore ritira sia pizze che bevande dalla cucina.

Fraasi relative a Bevanda

- I prodotti... sono pizze o bevande;
- Il barman può fare lo stesso per preparare le bevande;
- Quando... una bevanda... è pronta,... il barman la indica come preparata;
- Il gestore ritira sia pizze che bevande dalla cucina.

Frase relative a Menu

- I prodotti, appartenenti ad un menu fisso definito dal gestore del locale, sono pizze o bevande.

Frase relative a Cameriere

- I camerieri prendono le comande andando al tavolo ed aggiungendo i prodotti richiesti all'elenco di quelli da preparare;
- I camerieri possono consultare dai loro terminali quali prodotti sono pronti e a quali tavoli vanno consegnati.

Frase relative a Pizzaiolo

- I pizzaioli possono consultare il sistema per sapere quali pizze devono essere preparate;
- Quando una pizza... è pronta, il pizzaiolo... la indica come preparata.

Frase relative a Barman

- Il barman può fare lo stesso per preparare le bevande;
- Quando... una bevanda... è pronta,... il barman la indica come preparata.

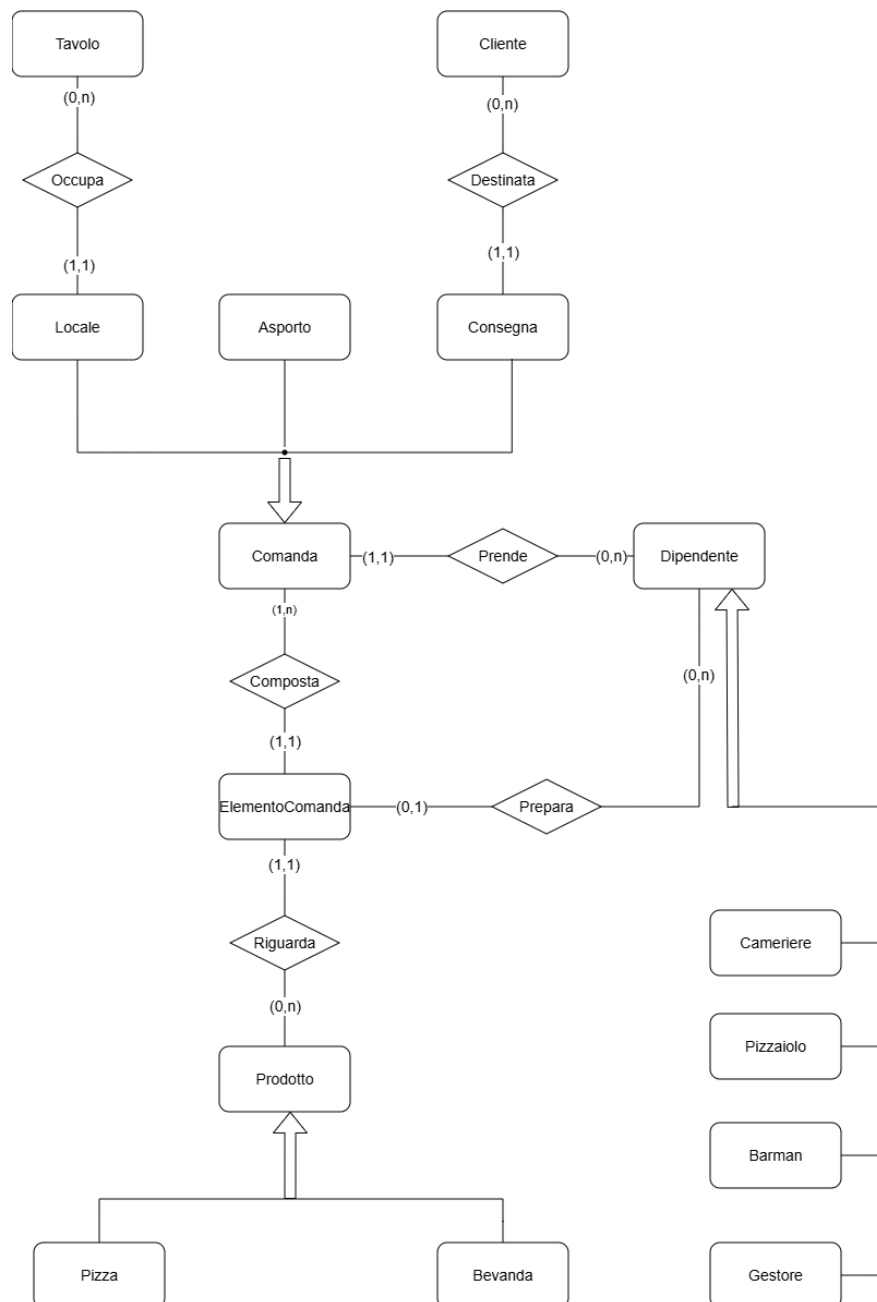
Frase relative a Gestore

- I prodotti, appartenenti ad un menu fisso definito dal gestore del locale, sono pizze o bevande;
- Il gestore registra l'indirizzo di consegna e il nome del cliente, oltre ad un recapito telefonico;
- Il gestore ritira sia pizze che bevande dalla cucina, per consegnarle al cliente o al fattorino;
- Il gestore segna la comanda come completata e può stampare lo scontrino non fiscale di riepilogo;
- Ha la possibilità di visualizzare le entrate giornaliere e/o mensili.

Frase relative a Fattorino
• Il gestore ritira sia pizze che bevande dalla cucina, per consegnarle al cliente o al fattorino.

3. Progettazione concettuale

Costruzione dello schema E-R



La costruzione dello schema è stata condotta con strategia **inside-out**, individuando un nucleo ristretto di concetti fondamentali ed estendendolo progressivamente verso l'esterno.

Passo 1 — Nucleo: comanda ed elemento di comanda. Il concetto centrale è la comanda, che compare in tutti gli insiemi di frasi raccolti nel capitolo precedente. La specifica attribuisce però lo stato di preparazione alla singola unità ordinata e non alla comanda nel suo complesso: «quando una

pizza o una bevanda (di una certa comanda) è pronta, il pizzaiolo o il barman la indicano come preparata». Se un cliente ordina due margherite, la prima può essere pronta mentre la seconda è in lavorazione, situazione che un semplice attributo *quantità* non catturerebbe. Si introducono quindi le entità *Comanda* ed *ElementoComanda*, legate dalla relazione *Composta*.

Poiché un elemento non ha senso al di fuori della comanda che lo contiene, *ElementoComanda* è identificata esternamente tramite *Comanda* più un numero progressivo. Le si attribuisce inoltre il *prezzoUnitario*: il gestore può modificare il menu in qualsiasi momento, ma lo scontrino deve riportare il prezzo vigente al momento dell'ordinazione.

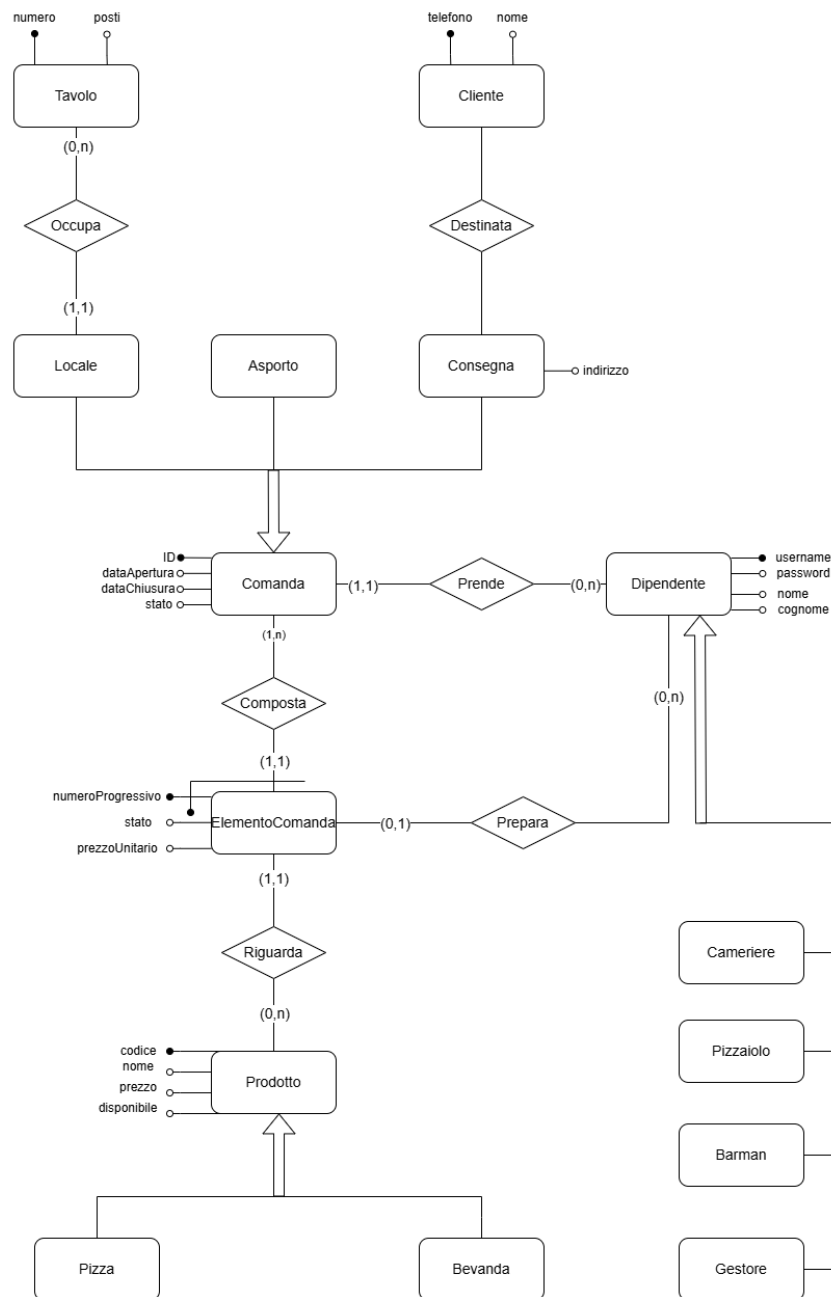
Passo 2 — Prodotto. Ogni elemento si riferisce a una voce del menu fisso, rappresentata dall'entità *Prodotto* e collegata dalla relazione *Riguarda*. Poiché la specifica afferma che i prodotti «sono pizze o bevande» e che le due categorie sono preparate da figure diverse, si introduce una generalizzazione totale ed esclusiva in *Pizza* e *Bevanda*. All'entità padre si aggiunge l'attributo *disponibile*, necessario perché un prodotto esaurito non può essere eliminato senza perdere lo storico delle comande in cui compare.

Passo 3 — Tavolo, cliente e modalità della comanda. La specifica prevede tre modalità mutuamente esclusive, ciascuna con dati diversi: il consumo sul posto richiede un tavolo, la consegna richiede i dati del cliente, l'asporto nessuno dei due. Collegare *Tavolo* e *Cliente* direttamente a *Comanda* comporterebbe due partecipazioni opzionali, che non impedirebbero l'esistenza di una comanda contemporaneamente assegnata a un tavolo e destinata a una consegna. Si introduce pertanto una generalizzazione totale ed esclusiva di *Comanda* in *Locale*, *Asporto* e *Consegna*, agganciando ciascuna relazione alla sola sottoentità pertinente.

Cliente è identificato dal recapito telefonico, unico dato che la specifica indica come registrato all'atto dell'ordine. L'indirizzo è invece attributo di *Consegna* e non di *Cliente*, poiché uno stesso cliente può richiedere consegne a indirizzi diversi. *Tavolo* partecipa con cardinalità (0,n) perché la relazione rappresenta lo storico delle comande servite e non l'occupazione istantanea.

Passo 4 — Dipendenti. I quattro ruoli individuati nella specifica sono accomunati dall'accesso al sistema: si introduce l'entità *Dipendente*, identificata dall'username, con generalizzazione totale ed esclusiva in *Cameriere*, *Pizzaiolo*, *Barman* e *Gestore*. Le relazioni *Prende* e *Prepara* registrano rispettivamente chi ha acquisito la comanda e chi ha segnato come preparato un elemento; entrambe sono agganciate all'entità padre, mentre i vincoli sui ruoli abilitati sono riportati tra le regole aziendali.

Integrazione finale



Conflitti sui nomi. La sinonimia tra *ordine* e *comanda*, e quella tra *proprietario*, *manager* e *gestore*, erano già state risolte in fase di disambiguazione. In fase di costruzione è invece emersa un'omonimia sul termine *prodotto*, usato dalla specifica sia per la voce di listino («i prodotti, appartenenti ad un menu fisso») sia per la singola unità da consegnare («quali prodotti sono pronti e a quali tavoli vanno consegnati»). Il conflitto è stato risolto riservando il nome *Prodotto* alla voce di menu e introducendo *ElementoComanda* per l'unità richiesta.

Conflitti strutturali. Il cliente compariva con struttura diversa nei due frammenti: entità con dati propri nelle frasi sulle consegne, semplice occupante di un tavolo in quelle sul consumo sul posto. Si è scelto di modellarlo come entità collegata alla sola sottoentità *Consegna*. Il secondo conflitto, relativo alle partecipazioni opzionali di *Tavolo* e *Cliente*, è stato risolto con la generalizzazione descritta al passo 3.

Concetti non modellati. Il *menu* non diventa un'entità autonoma, avendo una sola occorrenza: l'insieme dei prodotti offerti coincide con le istanze di *Prodotto* disponibili. Il *fattorino* non compare nello schema in quanto non interagisce con il sistema, essendo citato dalla specifica solo come destinatario fisico della consegna da parte del gestore.

Cicli. Il ciclo tra *Dipendente*, *Comanda* ed *ElementoComanda* è giustificato: chi prende la comanda e chi ne prepara gli elementi appartengono per definizione a ruoli diversi, quindi nessuna delle tre relazioni è deducibile dalle altre.

Regole aziendali

- Un elemento relativo a una pizza può essere segnato come preparato solo da un pizzaiolo, uno relativo a una bevanda solo da un barman.
- Una comanda locale può essere presa solo da un cameriere, una da asporto o in consegna solo dal gestore.
- Un tavolo può essere associato ad al più una comanda non completata per volta.
- Una comanda può essere completata solo se tutti i suoi elementi risultano consegnati.
- Non è possibile aggiungere, modificare o rimuovere elementi di una comanda completata.
- Un elemento può riferirsi solo a un prodotto disponibile al momento dell'inserimento.
- Solo il gestore può modificare il menu, completare le comande, stampare gli scontrini e consultare le entrate.
- Il totale di una comanda è la somma dei prezzi unitari dei suoi elementi; le entrate di un periodo sono la somma dei totali delle comande completate nel periodo.
- Un tavolo è libero se non esiste alcuna comanda locale a esso associata con stato diverso da completata

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
--------	-------------	-----------	----------------

Comanda	Insieme dei prodotti richiesti da un cliente in un'unica ordinazione, dalla presa in carico al completamento	dataApertura, dataChiusura, stato	ID
Locale	Comanda consumata sul posto, servita presso un tavolo del locale		ID (da Comanda)
Asporto	Comanda ritirata direttamente dal cliente presso il locale		ID (da Comanda)
Consegna	Comanda acquisita telefonicamente e recapitata all'indirizzo del cliente	indirizzo	ID (da Comanda)
ElementoComanda	Singola unità di prodotto richiesta all'interno di una comanda, dotata di un proprio stato di preparazione	numeroProgressivo, stato, prezzoUnitario	numeroProgressivo + Comanda
Prodotto	Voce del menu fisso del locale, richiedibile all'interno di una comanda	Nome, prezzo, disponibile	codice
Pizza	Prodotto la cui preparazione compete al pizzaiolo		codice (da Prodotto)
Bevanda	Prodotto la cui preparazione compete al barman		codice (da Prodotto)
Tavolo	Postazione del locale presso cui viene servita una comanda consumata sul posto	numero, posti	numero
Cliente	Persona che effettua una comanda in consegna, di cui il sistema registra i dati di recapito	telefono, nome	telefono
Dipendente	Lavoratore della pizzeria dotato di credenziali di accesso al sistema	username, password, nome, cognome	username
Cameriere	Dipendente che assegna i tavoli, prende le comande consumate sul posto e consegna i prodotti pronti		username (da Dipendente)
Pizzaiolo	Dipendente che consulta le pizze da preparare e le segna come pronte		username (da Dipendente)
Barman	Dipendente che consulta le bevande da preparare e le segna come pronte		username (da Dipendente)
Gestore	Dipendente che definisce il menu, acquisisce le comande da asporto e in consegna, completa le comande e consulta le statistiche		username (da Dipendente)

4. Progettazione logica

Volume dei dati

- Si considera una pizzeria di dimensioni medie: **20 tavoli**, **7 dipendenti** (1 gestore, 3 camerieri, 2 pizzaioli, 1 barman) con turnover di 2 unità l'anno, e un menu di **50 prodotti** attivi (30 pizze e 20 bevande) di cui una decina sostituiti ogni anno. I prodotti dismessi non vengono eliminati, ma resi non disponibili, per non perdere lo storico delle comande.
- Operando sette giorni su sette a pranzo e a cena, il locale serve **70 comande al giorno**: 45 sul posto (15 a pranzo e 30 a cena, con 1,5 turni per tavolo la sera), 10 da asporto e 15 in consegna. Ogni comanda comprende in media **5 elementi**, di cui 3 pizze e 2 bevande. Si registrano solo i clienti delle consegne, come previsto dalla specifica, e si stima che ciascuno ordini in media 10 volte. I dati sono conservati per **5 anni** ai fini delle elaborazioni statistiche richieste.
- Dipendente: $7 + 2 \times 5 = 17$
- Prodotto: $50 + 10 \times 5 = 100$
- Cliente: $(15 \times 365 \times 5) / 10 \approx 2.800$
- Comanda: $70 \times 365 \times 5 = 127.750$
- ElementoComanda: $70 \times 5 \times 365 \times 5 = 638.750$
- Le relazioni hanno volume pari al numero di occorrenze dell'entità che vi partecipa con cardinalità massima unitaria

Concetto nello schema	Tipo ¹	Volume atteso
Tavolo	E	20
Cliente	E	2800
Dipendente	E	17
Cameriere	E	8
Pizzaiolo	E	5
Barman	E	3
Gestore	E	1
Prodotto	E	100
Pizza	E	60
Bevanda	E	40
Comanda	E	127.750
Locale	E	82.125
Asporto	E	18.250
Consegna	E	27.375

¹ Indicare con E le entità, con R le relazioni

ElementoComanda	E	638.750
Occupa	R	82.125
Destinata	R	27.375
Prende	R	127.750
Composta	R	638.750
Riguarda	R	638.750
Prepara	R	638.750

Tavola delle operazioni

Di seguito i calcoli per la frequenza attesa:

- **Login:** ogni dipendente accede all'inizio del proprio turno, 2 volte al giorno considerando i due servizi: $7 \times 2 = 14/\text{gg}$
- **Assegna tavolo e apri comanda:** una per ogni comanda consumata sul posto = $45/\text{gg}$
- **Aggiungi elemento a comanda locale:** $45 \times 5 = 225/\text{gg}$
- **Visualizza elementi pronti da servire:** si suppone che ciascuno dei 3 camerieri consulti il terminale una quarantina di volte per servizio = $120/\text{gg}$
- **Segna elemento come consegnato al tavolo:** $225/\text{gg}$
- **Visualizza pizze da preparare:** i 2 pizzaioli consultano il sistema prima di ogni infornata, mediamente 50 volte ciascuno = $100/\text{gg}$
- **Segna pizza come preparata:** $70 \times 3 = 210/\text{gg}$
- **Visualizza bevande da preparare:** il barman consulta il sistema circa 60 volte = $60/\text{gg}$
- **Segna bevanda come preparata:** $70 \times 2 = 140/\text{gg}$
- **Apri comanda da asporto:** $10/\text{gg}$
- **Registra comanda telefonica:** $15/\text{gg}$
- **Aggiungi elemento a comanda da asporto o in consegna:** $25 \times 5 = 125/\text{gg}$
- **Visualizza elementi pronti da ritirare:** il gestore controlla la cucina circa 50 volte = $50/\text{gg}$
- **Completa comanda:** una per comanda = $70/\text{gg}$
- **Stampa scontrino:** $70/\text{gg}$
- **Visualizza entrate giornaliere:** una volta a fine giornata = $1/\text{gg}$
- **Visualizza entrate mensili:** $12/\text{aa}$
- **Inserisci o modifica un prodotto del menu:** $10/\text{aa}$
- **Rendi non disponibile un prodotto:** si stima un esaurimento temporaneo un paio di volte al mese = $20/\text{aa}$

- **Registra o rimuovi un dipendente: 4/aa**

Cod.	Descrizione	Frequenza attesa
L1	Login	14/gg
C1	Assegna tavolo e apri comanda locale	45/gg
C2	Aggiungi elemento a comanda locale	225/gg
C3	Visualizza elementi pronti da servire ai tavoli	120/gg
C4	Segna elemento come consegnato al tavolo	225/gg
P1	Visualizza pizza da preparare	100/gg
P2	Segna pizza come preparata	210/gg
B1	Visualizza bevanda da preparare	60/gg
B2	Segna bevanda come preparata	140/gg
G1	Apri comanda da asporto	10/gg
G2	Registra comanda telefonica in consegna	15/gg
G3	Aggiungi elemento a comanda da asporto in consegna	125/gg
G4	Visualizza elementi pronti da ritirare	50/gg
G5	Completa comanda	70/gg
G6	Stampa scontrino non fiscale	70/gg
G7	Visualizza entrate giornaliere	1/gg
G8	Visualizza entrate mensili	12/aa
G9	Inserisci o modifica un prodotto del menu	30/aa
G10	Registra o rimuovi un dipendente	4/aa

Costo delle operazioni

Si assegna peso 1 agli accessi in lettura e peso 2 a quelli in scrittura. Per le consultazioni si considerano le occorrenze effettivamente coinvolte in un dato istante, gli elementi in attesa sono solo quelli delle comande aperte e non il volume complessivo del concetto.

1. L1: Login (14/gg)

- Lettura: Dipendente 1
- Costo per operazione: 1 unità $\rightarrow 1 \times 14 = 14/\text{gg}$

2. C1: Assegna tavolo e apri comanda locale (45/gg)

- Lettura: Tavolo 20, Comanda 20. Scrittura: Comanda 1, Occupa 1
- Costo per operazione: 44 unità $\rightarrow 44 \times 45 = 1.980/\text{gg}$

- L'individuazione di un tavolo libero richiede di confrontare i tavoli con le comande locali aperte, non essendo previsto un attributo di stato sul tavolo

3. C2: Aggiungi elemento a comanda locale (225/gg)

- Lettura: Prodotto 1, Comanda 1. Scrittura: ElementoComanda 1, Composta 1, Riguarda 1
- Costo per operazione: 8 unità $\rightarrow 8 \times 225 = 1.800/\text{gg}$

4. C3: Visualizza elementi pronti da servire (120/gg)

- Lettura: ElementoComanda 15, Prodotto 15, Comanda 15, Occupa 15, Tavolo 15
- Costo per operazione: 75 unità $\rightarrow 75 \times 120 = 9.000/\text{gg}$

5. C4: Segna elemento come consegnato al tavolo (225/gg)

- Lettura: ElementoComanda 1. Scrittura: ElementoComanda 1
- Costo per operazione: 3 unità $\rightarrow 3 \times 225 = 675/\text{gg}$

6. P1: Visualizza pizze da preparare (100/gg)

- Lettura: ElementoComanda 20, Riguarda 20, Pizza 20
- Costo per operazione: 60 unità $\rightarrow 60 \times 100 = 6.000/\text{gg}$

7. P2: Segna pizza come preparata (210/gg)

- Lettura: ElementoComanda 1. Scrittura: ElementoComanda 1, Prepara 1
- Costo per operazione: 5 unità $\rightarrow 5 \times 210 = 1.050/\text{gg}$

8. B1: Visualizza bevande da preparare (60/gg)

- Lettura: ElementoComanda 10, Riguarda 10, Bevanda 10
- Costo per operazione: 30 unità $\rightarrow 30 \times 60 = 1.800/\text{gg}$

9. B2: Segna bevanda come preparata (140/gg)

- Lettura: ElementoComanda 1. Scrittura: ElementoComanda 1, Prepara 1
- Costo per operazione: 5 unità $\rightarrow 5 \times 140 = 700/\text{gg}$

10. G1: Apri comanda da asporto (10/gg)

- Scrittura: Comanda 1, Prende 1
- Costo per operazione: 4 unità $\rightarrow 4 \times 10 = 40/\text{gg}$

11. G2: Registra comanda telefonica in consegna (15/gg)

- Lettura: Cliente 1. Scrittura: Cliente 0,4, Comanda 1, Prende 1, Destinata 1
- Costo per operazione: $7,8 \approx 8$ unità $\rightarrow 8 \times 15 = 120/\text{gg}$
- Il cliente viene cercato per recapito telefonico e inserito solo se nuovo, stimato nel 40% dei casi

12. G3: Aggiungi elemento a comanda da asporto o in consegna (125/gg)

- Lettura: Prodotto 1, Comanda 1. Scrittura: ElementoComanda 1, Composta 1, Riguarda 1
- Costo per operazione: 8 unità $\rightarrow 8 \times 125 = 1.000/\text{gg}$

13. G4: Visualizza elementi pronti da ritirare (50/gg)

- Lettura: ElementoComanda 8, Riguarda 8, Prodotto 8, Comanda 8
- Costo per operazione: 32 unità $\rightarrow 32 \times 50 = 1.600/\text{gg}$

14. G5: Completa comanda (70/gg)

- Lettura: ElementoComanda 5. Scrittura: Comanda 1
- Costo per operazione: 7 unità $\rightarrow 7 \times 70 = 490/\text{gg}$

15. G6: Stampa scontrino non fiscale (70/gg)

- Lettura: Comanda 1, ElementoComanda 5, Riguarda 5, Prodotto 5
- Costo per operazione: 16 unità $\rightarrow 16 \times 70 = 1.120/\text{gg}$

16. G7: Visualizza entrate giornaliere (1/gg)

- Lettura: Comanda 70, ElementoComanda 350
- Costo per operazione: 420 unità $\rightarrow 420 \times 1 = 420/\text{gg}$

17. G8: Visualizza entrate mensili (12/aa)

- Lettura: Comanda 2.100, ElementoComanda 10.500
- Costo per operazione: 12.600 unità $\rightarrow 12.600 \times 12 = 151.200/\text{aa} \approx 414/\text{gg}$

18. G9: Inserisci o modifica un prodotto del menu (30/aa)

- Lettura: Prodotto 1. Scrittura: Prodotto 1
- Costo per operazione: 3 unità $\rightarrow 3 \times 30 = 90/\text{aa}$

19. G10: Registra o rimuovi un dipendente (4/aa)

- Scrittura: Dipendente 1
- Costo per operazione: 2 unità $\rightarrow 2 \times 4 = 8/\text{aa}$

Le operazioni più onerose non sono quelle di scrittura ma le consultazioni ripetute di sala e cucina: C3 e P1 da sole assorbono circa 15.000 accessi al giorno. Seguono le statistiche G7 e G8, che pur avendo frequenza minima hanno costo unitario elevato per via della scansione degli elementi di tutte le comande del periodo.

Ristrutturazione dello schema E-R**Analisi delle ridondanze**

Attributo *totale* su Comanda — mantenuto. G7 e G8 devono sommare i prezzi degli elementi di tutte le comande del periodo, con un costo di 834 accessi al giorno. Un attributo derivato *totale*, calcolato al completamento della comanda, riduce entrambe le operazioni alla lettura delle sole comande: 139 accessi al giorno. Il costo aggiuntivo in scrittura è nullo, poiché G5 aggiorna comunque la riga della comanda e ne legge già gli elementi; l'occupazione aggiuntiva è di circa mezzo megabyte sull'intero orizzonte quinquennale.

Attributo *stato* su Tavolo — mantenuto. C1 deve confrontare i 20 tavoli con le comande locali aperte, per 44 accessi a esecuzione. Con uno stato esplicito il costo scende a 24 accessi, cui si aggiungono 2 scritture in G5 per liberare il tavolo: 1.170 accessi al giorno contro 1.980, a fronte di 20 byte di memoria.

Attributo *nomeProdotto* su ElementoComanda — scartato. Eliminarebbe circa 5.150 accessi al giorno in C3, P1, B1, G4 e G6, ma il risparmio è solo apparente: *Prodotto* conta 100 occorrenze e risiederà stabilmente in cache. L'attributo dipenderebbe inoltre dal codice del prodotto, che non è chiave dell'elemento, portando il modello fuori dalla terza forma normale, con 25 MB di occupazione aggiuntiva.

Si precisa che *prezzoUnitario* non costituisce una ridondanza: non replica il prezzo corrente del prodotto, ma registra quello vigente all'ordinazione, che può differirne dopo una modifica del menu.

Eliminazione delle generalizzazioni

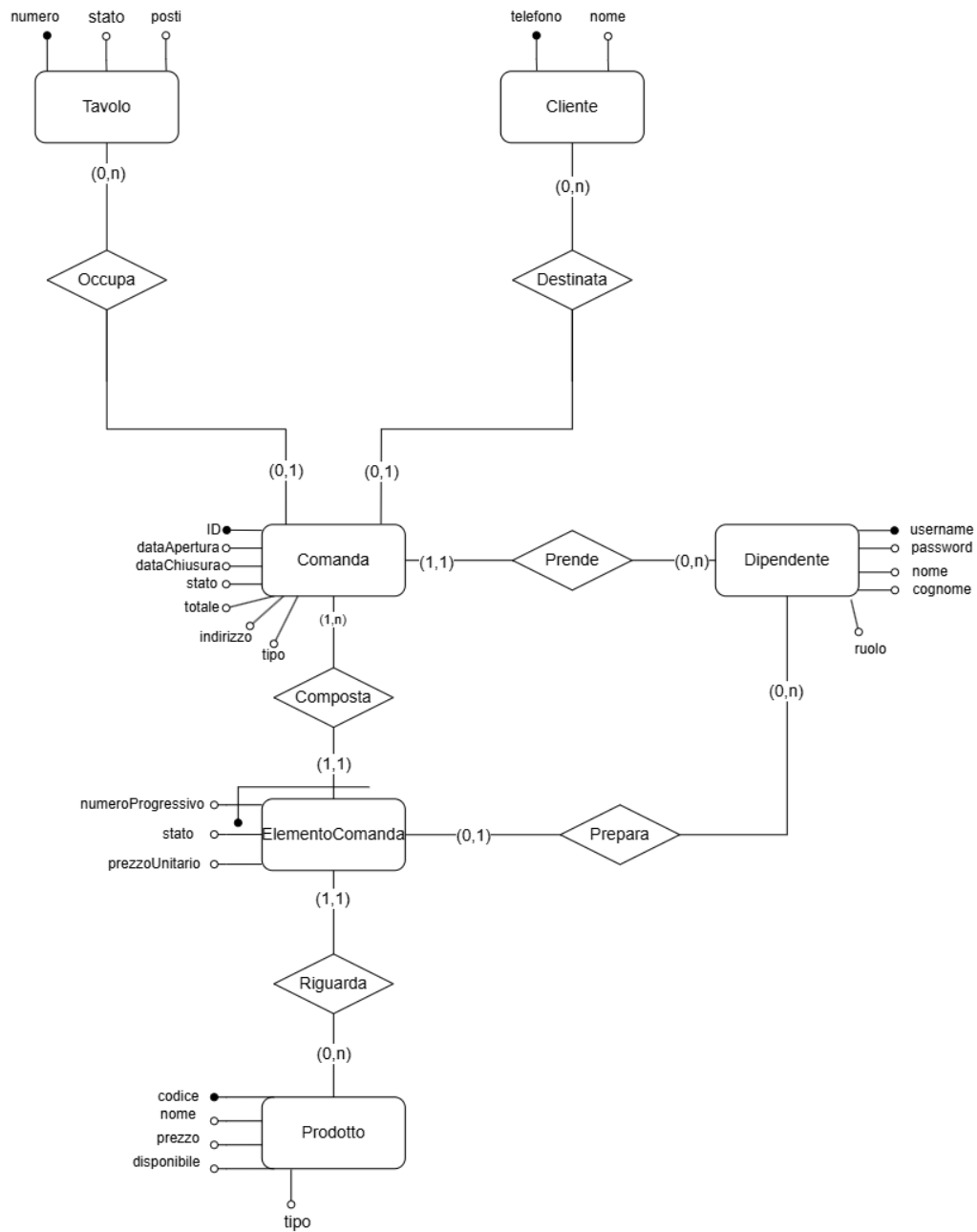
Prodotto. Le figlie non hanno attributi propri e solo P1 e B1 operano su una sola categoria, mentre C2, G3 e G6 accedono al prodotto indistintamente. Si accorpa nel padre con l'attributo *tipo* $\in \{\text{PIZZA}, \text{BEVANDA}\}$, imponendo una selezione aggiuntiva alle sole due operazioni di consultazione anziché un'unione a tutte le altre.

Dipendente. Le figlie non hanno attributi propri e le relazioni *Prende* e *Prepara* insistono già sul padre. Si accorpa nel padre con l'attributo *ruolo*; i vincoli sui ruoli abilitati saranno realizzati mediante il sistema dei privilegi del DBMS e, dove i ruoli condividono le stesse procedure, mediante verifica interna alle procedure stesse.

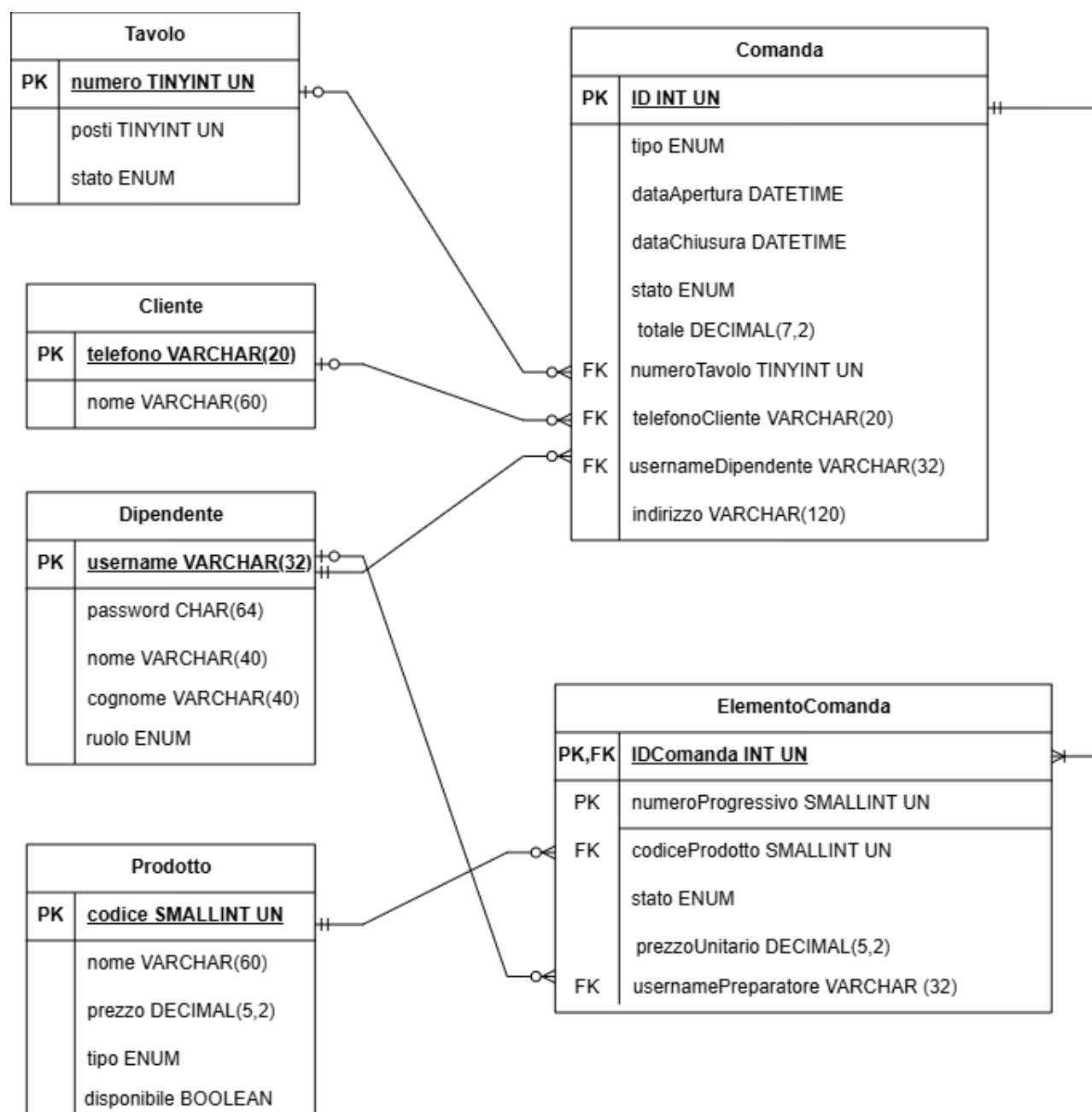
Comanda. Mantenere le tre figlie separate imporrebbe un'unione a tre vie a C2, G3, G5, G6, G7 e G8, ovvero proprio alle operazioni più costose, e una tripla traduzione della relazione *Composta*. Si accorpa nel padre con l'attributo *tipo* $\in \{\text{LOCALE}, \text{ASPORTO}, \text{CONSEGNA}\}$, accettando tre attributi opzionali, riferimento al tavolo, riferimento al cliente e indirizzo, la cui coerenza con il tipo sarà garantita da un vincolo di CHECK

Scelta degli identificatori primari

Comanda conserva l'*ID* generato dal sistema, preferito a una chiave composta da data e progressivo giornaliero perché quest'ultima si propagherebbe come chiave esterna sull'entità con il volume maggiore. *ElementoComanda* conserva l'identificatore esterno: il numero progressivo serve comunque a ordinare gli elementi e nessuna entità dipende da esso, per cui la chiave composta non si propaga oltre. *Prodotto* conserva il *codice* anziché il nome, modificabile dal gestore. *Tavolo*, *Cliente* e *Dipendente* conservano numero, recapito telefonico e username.



Di seguito il modello a tabelle:



Trasformazione di attributi e identificatori

Lo schema non presenta attributi ripetuti. L'unico attributo composto è l'*indirizzo* di consegna, scomponibile in via, civico, CAP e città: si è scelto di trattarlo come atomico, poiché il sistema non deve mai interrogare o aggregare i dati per singola componente ma solo registrarli e rileggerli integralmente.

L'unico identificatore esterno è quello di *ElementoComanda*, trasformato in chiave primaria composta affiancando al numero progressivo la chiave esterna verso la comanda. La dipendenza esistenziale è resa con una politica di cancellazione a cascata.

Traduzione di entità e associazioni

Tutte le associazioni hanno cardinalità massima unitaria da un lato e sono quindi tradotte mediante chiavi esterne, senza necessità di tabelle dedicate: *Occupi*, *Destinata* e *Prende* su *Comanda*, *Composta*, *Riguarda* e *Prepara* su *ElementoComanda*.

- **Tavolo**(numero, posti, stato)
- **Cliente**(telefono, nome)
- **Dipendente**(username, password, nome, cognome, ruolo)
- **Prodotto**(codice, nome, prezzo, tipo, disponibile)
- **Comanda**(ID, tipo, dataApertura, dataChiusura, stato, totale, numeroTavolo, telefonoCliente, indirizzo, usernameDipendente)
- **ElementoComanda**(IDComanda, numeroProgressivo, codiceProdotto, stato, prezzoUnitario, usernamePreparatore)

Vincoli di integrità referenziale: Comanda.numeroTavolo → Tavolo.numero, Comanda.telefonoCliente → Cliente.telefono, Comanda.usernameDipendente → Dipendente.username, ElementoComanda.IDComanda → Comanda.ID, ElementoComanda.codiceProdotto → Prodotto.codice, ElementoComanda.usernamePreparatore → Dipendente.username.

Normalizzazione del modello relazionale

Tutti gli attributi sono atomici e ogni relazione ha una chiave primaria, quindi il modello è in **1NF**.

L'unica chiave composta è quella di *ElementoComanda*: codice del prodotto, stato, prezzo unitario e preparatore variano da elemento a elemento e non dipendono né dalla sola comanda né dal solo numero progressivo. Le altre relazioni hanno chiave semplice. Il modello è in **2NF**.

Non esistono dipendenze transitive. In *Comanda* l'indirizzo non dipende dal cliente, potendo lo stesso cliente ordinare a indirizzi diversi, e il *totale* è determinato dall'insieme degli elementi e non da un attributo della comanda. In *ElementoComanda* il *prezzoUnitario* non dipende dal codice del prodotto,

registrando il prezzo storico: è proprio per preservare questa proprietà che è stata scartata la ridondanza sul nome del prodotto. Il modello è in **3NF**.

Ogni determinante è una superchiave, per cui il modello soddisfa anche la **BCNF** e non richiede ulteriori decomposizioni.

I valori nulli introdotti dall'accorpamento della generalizzazione di *Comanda* non violano le forme normali, che riguardano le dipendenze funzionali e non l'obbligatorietà degli attributi; richiedono però un controllo di coerenza tra il tipo della comanda e gli attributi valorizzati, realizzato in fase di progettazione fisica mediante un vincolo di CHECK.

5. Progettazione fisica

Il DBMS di riferimento è MariaDB 10.6. Prima di descrivere le singole strutture è opportuno esplicitare il criterio architetturale adottato, dal quale discendono le scelte successive.

Nessun utente applicativo possiede privilegi di scrittura diretta sulle tabelle: gli sono concessi unicamente privilegi di esecuzione sulle stored procedure di sua competenza. Poiché le procedure sono definite con SQL SECURITY DEFINER, esse operano con i privilegi del proprietario dello schema, mentre l'utente chiamante non può aggirarle scrivendo direttamente. Ne consegue che **ogni scrittura sulla base di dati passa necessariamente per una procedura**, e che i controlli collocati al suo interno non sono bypassabili.

Questa impostazione permette di realizzare la logica applicativa nelle procedure anziché nei trigger, con due vantaggi. Il primo è che una procedura può aprire una transazione e acquisire i lock necessari prima di decidere se scrivere, mentre un trigger interviene quando la decisione di scrivere è già stata presa e non può quindi impedire condizioni di corsa. Il secondo è che una procedura restituisce al chiamante un errore diagnostico specifico, mentre un trigger può solo abortire l'operazione.

Un esempio concreto del primo vantaggio è offerto dall'operazione di aggiunta di un prodotto a una comanda, che deve calcolare il numero progressivo successivo mediante una funzione di aggregazione: si tratta di una lettura di intervallo, come tale esposta all'anomalia di lettura fantasma. Collocando il controllo nella procedura è possibile acquisire un lock esclusivo sulla singola riga della comanda e restare al livello di isolamento *repeatable read*, serializzando i soli inserimenti relativi alla stessa comanda. Se il medesimo controllo fosse stato collocato in un trigger, che non può acquisire lock espliciti, l'unico modo di prevenire l'anomalia sarebbe stato elevare l'isolamento a *serializable*, con il conseguente blocco di intervallo sull'intera tabella degli elementi e una drastica riduzione del parallelismo.

I trigger restano riservati alle sole asserzioni che devono valere indipendentemente dal percorso di scrittura e che coinvolgono più tabelle, non essendo quindi esprimibili tramite vincoli di CHECK.

Utenti e privilegi

Sono previsti cinque utenti applicativi, più il proprietario dello schema utilizzato come definer. Quattro corrispondono ai ruoli individuati nella specifica; ad essi si aggiunge **app_login**, impiegato dall'applicazione nella fase precedente all'autenticazione, quando il ruolo del dipendente non è ancora

noto e non è quindi possibile connettersi con le sue credenziali. A tale utente è concesso il solo privilegio di esecuzione su `sp_login`, senza alcun accesso ai dati; subito dopo il login la connessione viene riaperta con le credenziali del ruolo.

app_cameriere: assegna i tavoli, prende le comande consumate sul posto e serve i prodotti pronti.

- Esecuzione: `sp_login`, `sp_apri_comanda_locale`, `sp_aggiungi_elemento`, `sp_elementi_pronti_sala`, `sp_segna_elemento_consegnato`, `sp_lista_menu`, `sp_tavoli_liberi`, `sp_comande_aperte`

app_pizzaiolo: consulta le pizze da preparare e le segna come pronte.

- Esecuzione: `sp_login`, `sp_coda_preparazione`, `sp_segna_elemento_preparato`

app_barman: consulta le bevande da preparare e le segna come pronte.

- Esecuzione: `sp_login`, `sp_coda_preparazione`, `sp_segna_elemento_preparato`

app_gestore: gestisce il menu, le comande da asporto e in consegna, la chiusura delle comande e le statistiche.

- Esecuzione: tutte le procedure del cameriere, più `sp_apri_comanda_asporto`, `sp_apri_comanda_consegna`, `sp_elementi_pronti_ritiro`, `sp_completa_comanda`, `sp_stampa_scontrino`, `sp_entrare_giornaliere`, `sp_entrare_mensili`, `sp_salva_prodotto`, `sp_registra_dipendente`, `sp_rimuovi_dipendente`

```
CREATE USER 'app_login'@'%' IDENTIFIED BY '...';
```

```
GRANT EXECUTE ON PROCEDURE pizzeria.sp_login TO 'app_login'@'%';
```

```
CREATE USER 'app_cameriere'@'%' IDENTIFIED BY '...';
```

```
CREATE USER 'app_pizzaiolo'@'%' IDENTIFIED BY '...';
```

```
CREATE USER 'app_barman'@'%' IDENTIFIED BY '...';
```

```
CREATE USER 'app_gestore'@'%' IDENTIFIED BY '...';
```

```
-- nessun GRANT su tabelle o viste: solo EXECUTE sulle procedure elencate
```

```
GRANT EXECUTE ON PROCEDURE pizzeria.sp_apri_comanda_locale TO 'app_cameriere'@'%';
```

```
-- ... (analogamente per le altre procedure e per gli altri utenti)
```

La distinzione tra pizzaiolo e barman non è affidata ai privilegi, poiché entrambi eseguono le medesime procedure: è **sp_segna_elemento_preparato** a verificare che il ruolo del dipendente corrisponda al tipo di prodotto, come previsto dalla regola aziendale 1.

Si osservi inoltre che nessun utente applicativo possiede il privilegio DELETE su alcuna tabella. L'unica cancellazione prevista dal sistema è quella effettuata dall'evento di pulizia dello storico, eseguito con i privilegi del proprietario dello schema.

Strutture di memorizzazione

Tutte le tabelle utilizzano il motore InnoDB, dichiarato esplicitamente in fase di creazione: è l'unico motore di MariaDB che supporta contemporaneamente le transazioni, il blocco a livello di riga richiesto dalle clausole FOR UPDATE e i vincoli di integrità referenziale su cui si fonda la traduzione delle associazioni.

Tabella <Tavolo>		
Colonna	Tipo di dato	Attributi ²
numero	TINYINT	PK, NN, UN
posti	TINYINT	NN, UN
stato	ENUM('LIBERO', OCCUPATO ')	NN, default, 'LIBERO '

Tabella <Cliente>		
Colonna	Tipo di dato	Attributi ³
telefono	VARCHAR(20)	PK, NN
nome	VARCHAR(60)	NN

Tabella <Dipendente>		
Colonna	Tipo di dato	Attributi ⁴

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

³ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁴ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

username	VARCHAR(32)	PK, NN
password	CHAR(64)	NN
nome	VARCHAR(40)	NN
cognome	VARCHAR(40)	NN
ruolo	ENUM('CAMERIERE','PIZZAILO','BARMAN','GESTORE')	NN

La password non è memorizzata in chiaro ma come digest SHA-256, calcolato dalla procedura di registrazione: il campo ha quindi dimensione fissa di 64 caratteri esadecimali.

Tabella <Prodotto>		
Colonna	Tipo di dato	Attributi ⁵
codice	SMALLINT	PK, NN, UN, AI
nome	VARCHAR(60)	NN, UQ
prezzo	DECIMAL(5,2)	NN
tipo	ENUM('PIZZA','BEVANDA')	NN
disponibile	BOOLEAN	NN, default TRUE

Il prezzo è di tipo DECIMAL e non a virgola mobile, per evitare errori di arrotondamento nella somma che produce il totale della comanda.

Tabella <Comanda>		
Colonna	Tipo di dato	Attributi ⁶
ID	INT	PK, NN, UN, AI
tipo	ENUM('LOCALE','ASPORTO','CONSEGNA')	NN
dataApertura	DATETIME	NN, default CURRENT_TIMESTAMP
dataChiusura	DATETIME	
stato	ENUM('APERTA','COMPLETATA')	NN, default 'APERTA'
totale	DECIMAL(7,2)	
numeroTavolo	TINYINT	FK, UN
telefonoCliente	VARCHAR(20)	FK
indirizzo	VARCHAR(120)	
usernameDipendente	VARCHAR(32)	FK, NN

⁵ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁶ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella <ElementoComanda>		
Colonna	Tipo di dato	Attributi ⁷
IDComanda	INT	PK, FK, NN, UN
numeroProgressivo	SMALLINT	PK, NN, UN
codiceProdotto	SMALLINT	FK, NN, UN
stato	ENUM('IN_ATTESA','PRONTO','CONSEGNATO')	NN, default 'IN_ATTESA'
prezzoUnitario	DECIMAL(5,2)	NN
usernamePreparatore	VARCHAR(32)	FK

La chiave esterna verso Comanda è dichiarata ON DELETE CASCADE, in coerenza con la dipendenza esistenziale individuata in fase di progettazione concettuale.

Vincoli di dominio

I valori nulli introdotti dall'accorpamento della generalizzazione di Comanda richiedono un controllo di coerenza con la modalità, come anticipato in fase di normalizzazione. Trattandosi di un vincolo interno alla singola riga, è esprimibile mediante un CHECK, applicato da MariaDB a partire dalla versione 10.2.1, senza necessità di ricorrere a un trigger.

```
ALTER TABLE Comanda ADD CONSTRAINT chk_modalita CHECK (
    (tipo = 'LOCALE' AND numeroTavolo IS NOT NULL
      AND telefonoCliente IS NULL AND indirizzo IS NULL)
  OR (tipo = 'ASPORTO' AND numeroTavolo IS NULL
      AND telefonoCliente IS NULL AND indirizzo IS NULL)
  OR (tipo = 'CONSEGNA' AND numeroTavolo IS NULL
      AND telefonoCliente IS NOT NULL AND indirizzo IS NOT NULL)
);
```

```
ALTER TABLE Comanda ADD CONSTRAINT chk_chiusura CHECK (
    (stato = 'APERTA' AND dataChiusura IS NULL AND totale IS NULL)
  OR (stato = 'COMPLETATA' AND dataChiusura IS NOT NULL AND totale IS NOT NULL));
```

```
ALTER TABLE Prodotto ADD CONSTRAINT chk_prezzo CHECK (prezzo > 0);
```

⁷ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

```
ALTER TABLE Tavolo ADD CONSTRAINT chk_posti CHECK (posti > 0);
```

Indici

Non si riportano gli indici generati automaticamente per chiavi primarie e chiavi esterne. Gli indici introdotti per ragioni prestazionali sono due, entrambi motivati dall'analisi dei costi della sezione precedente

Tabella <ElementoComanda>	
Indice <idx elemento stato>	Tipo ⁸ : IDX
Colonna 1	stato

Le operazioni C3, P1, B1 e G4 assorbono da sole circa 18.000 accessi al giorno e consistono tutte nel selezionare gli elementi in un determinato stato. In assenza di indice richiederebbero una scansione completa di una tabella che raggiunge le 638.750 righe, mentre gli elementi effettivamente in attesa o pronti sono poche decine in ogni istante: l'indice riduce l'accesso alle sole righe pertinenti.

Tabella <Comanda>	
Indice <idx_comanda_stato_chiusura>	Tipo ⁹ : IDX
Colonna 1	stato
Colonna 2	dataChiusura

Le operazioni statistiche G7 e G8 selezionano le comande completate in un intervallo di date. L'indice composto, con lo stato come prima colonna e la data di chiusura come seconda, consente di individuare direttamente le comande del periodo anziché scandire le 127.750 righe della tabella. L'ordine delle colonne è determinato dal fatto che entrambe le operazioni filtrano sempre su entrambi i campi.

Perché l'ottimizzatore possa effettivamente impiegare questo indice, le due procedure statistiche esprimono il filtro temporale come confronto tra intervalli di date anziché applicando funzioni di estrazione alla colonna: una condizione della forma `DATE(dataChiusura) = ...` renderebbe l'indice inutilizzabile, obbligando comunque alla scansione completa.

Non si è ritenuto necessario alcun indice su Prodotto: con appena 100 righe la tabella risiede stabilmente in memoria, come già osservato scartando la ridondanza sul nome del prodotto.

⁸ IDX = index, UQ = unique, FT = full text, PR = primary.

⁹ IDX = index, UQ = unique, FT = full text, PR = primary.

Trigger

Coerentemente con il criterio esposto in apertura, i trigger sono limitati alle asserzioni che coinvolgono più tabelle e devono valere indipendentemente dalla procedura che effettua la scrittura. Ne sono stati implementati due, entrambi a protezione della regola aziendale 5, secondo cui una comanda completata non è più modificabile.

Si tratta di un vincolo che non può essere espresso con un CHECK, poiché nel primo caso lo stato della comanda risiede in una tabella diversa da quella degli elementi, e che è opportuno collocare a livello di schema anziché nelle procedure: esso protegge infatti l'integrità storica dei dati contabili, e deve continuare a valere anche per procedure che venissero aggiunte in futuro.

```
DELIMITER $$
```

```
DROP TRIGGER IF EXISTS trg_elemento_no_update$$
```

```
CREATE TRIGGER trg_elemento_no_update
```

```
BEFORE UPDATE ON ElementoComanda
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF (SELECT stato FROM Comanda WHERE ID = OLD.IDComanda) = 'COMPLETATA' THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Impossibile modificare elementi di una comanda completata';
```

```
    END IF;
```

```
END$$
```

```
DROP TRIGGER IF EXISTS trg_comanda_no_update$$
```

```
CREATE TRIGGER trg_comanda_no_update
```

```
BEFORE UPDATE ON Comanda
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF OLD.stato = 'COMPLETATA' THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Impossibile modificare una comanda completata';
```

```
    END IF;
```

END\$\$

DELIMITER ;

Il secondo trigger consente la transizione da APERTA a COMPLETATA, controllando lo stato precedente e non quello nuovo, e blocca ogni modifica successiva.

Non è stato invece introdotto alcun trigger sulla cancellazione degli elementi. La protezione contro le cancellazioni indebite è già garantita da due condizioni indipendenti: nessun utente applicativo possiede il privilegio DELETE sulle tabelle, e nessuna procedura effettua cancellazioni su ElementoComanda. Un trigger in tal senso interferirebbe inoltre con la politica di conservazione descritta nel paragrafo seguente, che rimuove i dati esauriti proprio mediante cancellazione a cascata.

Eventi

È previsto un unico evento, istanziato in fase di configurazione del sistema e mai creato in maniera effimera durante l'esecuzione delle procedure.

-- Richiede privilegi di amministrazione

```
SET GLOBAL event_scheduler = ON;
```

```
CREATE EVENT ev_pulizia_storico
```

```
ON SCHEDULE EVERY 1 MONTH
```

```
DO
```

```
DELETE FROM Comanda
```

```
WHERE stato = 'COMPLETATA'
```

```
AND dataChiusura < CURDATE() - INTERVAL 5 YEAR;
```

L'evento realizza la politica di conservazione quinquennale assunta nel calcolo dei volumi: senza di esso la base di dati crescerebbe indefinitamente e le stime della sezione precedente perderebbero validità. La cancellazione degli elementi associati avviene per effetto della cascata sulla chiave esterna, senza necessità di una seconda istruzione.

Viste

Sono definite tre viste, il cui scopo è centralizzare logiche di join e di selezione utilizzate da più procedure, evitando di replicarle in ciascuna di esse. Non vengono concessi privilegi diretti sulle viste: esse sono accessibili unicamente dalle procedure che le interrogano.

```
CREATE VIEW v_coda_preparazione AS
```

```
SELECT e.IDComanda, e.numeroProgressivo, p.nome AS prodotto, p.tipo,
```

```
c.tipo AS modalita, c.numeroTavolo, c.dataApertura
FROM ElementoComanda e
JOIN Prodotto p ON e.codiceProdotto = p.codice
JOIN Comanda c ON e.IDComanda = c.ID
WHERE e.stato = 'IN_ATTESA';
```

Utilizzata da P1 e B1, che ne selezionano rispettivamente le righe di tipo pizza e bevanda. Restituisce la data di apertura della comanda per consentire di rispettare l'ordine di arrivo.

```
CREATE VIEW v_elementi_pronti AS
SELECT e.IDComanda, e.numeroProgressivo, p.nome AS prodotto,
       c.tipo AS modalita, c.numeroTavolo
FROM ElementoComanda e
JOIN Prodotto p ON e.codiceProdotto = p.codice
JOIN Comanda c ON e.IDComanda = c.ID
WHERE e.stato = 'PRONTO';
```

Utilizzata da C3, che ne seleziona le comande locali per conoscere il tavolo di destinazione, e da G4, che seleziona quelle da asporto e in consegna.

```
CREATE VIEW v_comande_completate AS
SELECT ID, dataChiusura, totale
FROM Comanda
WHERE stato = 'COMPLETATA';
```

Utilizzata da G7 e G8 come base per il calcolo degli incassi. Si è scelto deliberatamente di non includere l'aggregazione nella vista: una vista contenente **GROUP BY** verrebbe materializzata in una tabella temporanea, e il filtro sul periodo sarebbe applicato soltanto dopo aver aggregato tutte le comande dell'intero storico, vanificando sia l'indice sia la ridondanza sull'attributo **totale**. Nella forma adottata la vista viene invece fusa nella query chiamante e il filtro temporale raggiunge direttamente l'indice. L'aggregazione è pertanto collocata nelle due procedure statistiche

Stored Procedures e transazioni

Le procedure sono state realizzate secondo tre criteri. Ciascuna corrisponde a **una singola operazione** della tavola delle operazioni ed è completa in sé: verifica le proprie precondizioni ed effettua le

scritture necessarie, senza richiedere al chiamante di orchestrare più chiamate. Ciascuna **verifica al proprio interno** i vincoli applicabili, sollevando un errore diagnostico specifico. Infine, le procedure che effettuano **più di una scrittura o che devono decidere sulla base di una lettura** sono realizzate come transazioni, acquisendo i lock necessari con `SELECT ... FOR UPDATE`.

Ogni procedura dichiara inoltre esplicitamente il livello di isolamento della propria transazione. Le procedure che modificano i dati adottano *repeatable read*, sufficiente perché la coerenza delle sezioni critiche è garantita da lock espliciti su singola riga acquisiti con `SELECT ... FOR UPDATE`, e non richiede quindi i blocchi di intervallo di *serializable*, che degraderebbero il parallelismo senza aggiungere garanzie. Le procedure di consultazione adottano invece *read committed* e dichiarano la transazione in sola lettura con `SET TRANSACTION READ ONLY`: non modificando alcun dato non sussiste rischio di corruzione dello stato, e il livello più permissivo massimizza il parallelismo del server.

Tutte le variabili destinate a ricevere il risultato di una `SELECT ... INTO` sono inizializzate a `NULL` in fase di dichiarazione: qualora l'interrogazione non restituisse alcuna riga, MariaDB si limita a emettere un avvertimento lasciando invariato il valore precedente, e l'inizializzazione esplicita garantisce che il successivo controllo di esistenza si comporti correttamente.

L1 — `sp_login`

Verifica le credenziali e restituisce il ruolo del dipendente. Il confronto avviene sul digest della password, mai trasmessa né memorizzata in chiaro.

```
DROP PROCEDURE IF EXISTS sp_login$$
```

```
CREATE PROCEDURE sp_login(  
  IN p_username VARCHAR(32),  
  IN p_password VARCHAR(64),  
  OUT p_ruolo  VARCHAR(16)  
)
```

```
BEGIN
```

```
  SET p_ruolo = NULL;
```

```
  SELECT ruolo INTO p_ruolo  
  FROM Dipendente
```

```
WHERE username = p_username
AND password = SHA2(p_password, 256);

IF p_ruolo IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Credenziali non valide';
END IF;
END;
```

C1 — sp_apri_comanda_locale

Assegna un tavolo libero e apre la comanda associata. È transazionale perché effettua due scritture e perché la verifica di disponibilità del tavolo deve essere protetta da condizioni di corsa: due camerieri potrebbero altrimenti assegnare lo stesso tavolo contemporaneamente. Il lock esclusivo sulla riga del tavolo serializza le richieste concorrenti, realizzando la regola aziendale 3.

```
DROP PROCEDURE IF EXISTS sp_apri_comanda_locale$$

CREATE PROCEDURE sp_apri_comanda_locale(
    IN p_numeroTavolo TINYINT UNSIGNED,
    IN p_username    VARCHAR(32),
    OUT p_idComanda  INT UNSIGNED
)
BEGIN
    DECLARE v_stato ENUM('LIBERO','OCCUPATO') DEFAULT NULL;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
    START TRANSACTION;

    SELECT stato INTO v_stato
    FROM Tavolo WHERE numero = p_numeroTavolo FOR UPDATE;

    IF v_stato IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tavolo inesistente';
    END IF;
```

```
IF v_stato = 'OCCUPATO' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tavolo già occupato';
END IF;
```

```
INSERT INTO Comanda (tipo, numeroTavolo, usernameDipendente)
VALUES ('LOCALE', p_numeroTavolo, p_username);
SET p_idComanda = LAST_INSERT_ID();
```

```
UPDATE Tavolo SET stato = 'OCCUPATO' WHERE numero = p_numeroTavolo;
```

```
COMMIT;
END;
```

C2, G3 — sp_aggiungi_elemento

Aggiunge un'unità di prodotto a una comanda aperta. La procedura è unica per camerieri e gestore, poiché l'operazione è identica e la differenza sta solo nella modalità della comanda, già fissata all'apertura.

È transazionale per tre ragioni concorrenti: verifica che la comanda sia ancora aperta (regola 5), verifica che il prodotto sia disponibile (regola 6) e calcola il numero progressivo successivo. Quest'ultimo punto è il più delicato: due camerieri che aggiungessero simultaneamente un elemento alla stessa comanda calcolerebbero lo stesso progressivo, violando la chiave primaria. Il lock esclusivo sulla riga della comanda serializza le aggiunte.

Si noti che il prezzo del prodotto viene copiato sull'elemento al momento dell'inserimento: è qui che si concretizza il congelamento del prezzo storico discusso in fase di progettazione concettuale.

```
DROP PROCEDURE IF EXISTS sp_aggiungi_elemento$$
```

```
CREATE PROCEDURE sp_aggiungi_elemento(
    IN p_idComanda    INT UNSIGNED,
    IN p_codiceProdotto SMALLINT UNSIGNED,
    OUT p_numero      SMALLINT UNSIGNED
)
```

```
BEGIN
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
    DECLARE v_statoComanda ENUM('APERTA','COMPLETATA') DEFAULT NULL;
    DECLARE v_disponibile BOOLEAN    DEFAULT NULL;
    DECLARE v_prezzo    DECIMAL(5,2) DEFAULT NULL;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
    START TRANSACTION;;

    SELECT stato INTO v_statoComanda
    FROM Comanda WHERE ID = p_idComanda FOR UPDATE;

    IF v_statoComanda IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Comanda inesistente';
    END IF;
    IF v_statoComanda = 'COMPLETATA' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Comanda gia completata';
    END IF;

    SELECT disponibile, prezzo INTO v_disponibile, v_prezzo
    FROM Prodotto WHERE codice = p_codiceProdotto;

    IF v_disponibile IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Prodotto inesistente';
    END IF;
    IF NOT v_disponibile THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Prodotto non disponibile';
    END IF;

    SELECT COALESCE(MAX(numeroProgressivo), 0) + 1 INTO p_numero
    FROM ElementoComanda WHERE IDComanda = p_idComanda;
```

```
INSERT INTO ElementoComanda
  (IDComanda, numeroProgressivo, codiceProdotto, prezzoUnitario)
VALUES (p_idComanda, p_numero, p_codiceProdotto, v_prezzo);

COMMIT;
END;
```

C3 — sp_elementi_pronti_sala

Restituisce i prodotti pronti da servire ai tavoli, con l'indicazione del tavolo di destinazione.

```
DROP PROCEDURE IF EXISTS sp_elementi_pronti_sala$$

CREATE PROCEDURE sp_elementi_pronti_sala()
BEGIN
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION READ ONLY;
START TRANSACTION;

  SELECT IDComanda, numeroProgressivo, prodotto, numeroTavolo
  FROM v_elementi_pronti
  WHERE modalita = 'LOCALE'
  ORDER BY numeroTavolo;

COMMIT;
END;
```

C4 — sp_segna_elemento_consegnato

Registra la consegna al tavolo. La verifica dello stato e l'aggiornamento sono racchiusi in un'unica istruzione condizionale, così che due camerieri non possano registrare due volte la stessa consegna: se nessuna riga viene aggiornata, la preconditione non era soddisfatta.

```
DROP PROCEDURE IF EXISTS sp_segna_elemento_consegnato$$

CREATE PROCEDURE sp_segna_elemento_consegnato(
```

```
IN p_idComanda INT UNSIGNED,

IN p_numero SMALLINT UNSIGNED

)

BEGIN

UPDATE ElementoComanda

SET stato = 'CONSEGNATO'

WHERE IDComanda = p_idComanda

AND numeroProgressivo = p_numero

AND stato = 'PRONTO';

IF ROW_COUNT() = 0 THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Elemento inesistente o non ancora pronto';

END IF;

END;
```

P1, B1 — sp_coda_preparazione

Restituisce i prodotti da preparare del tipo richiesto, in ordine di arrivo delle comande. La procedura è unica per pizzaiolo e barman, parametrizzata sul tipo di prodotto.

```
DROP PROCEDURE IF EXISTS sp_coda_preparazione$$
```

```
CREATE PROCEDURE sp_coda_preparazione(
```

```
    IN p_tipo ENUM('PIZZA','BEVANDA')
```

```
)
```

```
BEGIN
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
SET TRANSACTION READ ONLY;
```

```
START TRANSACTION;
```

```
    SELECT IDComanda, numeroProgressivo, prodotto, modalita, numeroTavolo
```

```
    FROM v_coda_preparazione
```

```
    WHERE tipo = p_tipo
```

```
    ORDER BY dataApertura, IDComanda, numeroProgressivo;
```

```
COMMIT;
```

```
END;
```

P2, B2 — sp_segna_elemento_preparato

Registra la preparazione di un elemento. È transazionale perché la decisione di scrivere dipende da due letture su tabelle diverse: lo stato dell'elemento e il ruolo del dipendente. La procedura realizza la regola aziendale 1, verificando che una pizza sia segnata come pronta solo da un pizzaiolo e una bevanda solo da un barman: il controllo non può essere delegato ai privilegi, poiché entrambi i ruoli eseguono questa stessa procedura. Si noti il controllo di esistenza del dipendente collocato prima del confronto sul ruolo: in sua assenza, un username inesistente lascerebbe la variabile a NULL e le disuguaglianze successive varrebbero a loro volta NULL, rendendo falsa la condizione e permettendo di eludere il controllo.

```
DROP PROCEDURE IF EXISTS sp_segna_elemento_preparato$$
```

```
CREATE PROCEDURE sp_segna_elemento_preparato(
```

```
    IN p_idComanda INT UNSIGNED,
```

```
    IN p_numero SMALLINT UNSIGNED,
```

```
    IN p_username VARCHAR(32)
```

```
)
```

BEGIN

DECLARE v_tipo ENUM('PIZZA','BEVANDA') DEFAULT NULL;

DECLARE v_stato ENUM('IN_ATTESA','PRONTO','CONSEGNATO') DEFAULT NULL;

DECLARE v_ruolo ENUM('CAMERIERE','PIZZAIOLO','BARMAN','GESTORE') DEFAULT NULL;

DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

START TRANSACTION;;

SELECT e.stato, p.tipo INTO v_stato, v_tipo

FROM ElementoComanda e

JOIN Prodotto p ON e.codiceProdotto = p.codice

WHERE e.IDComanda = p_idComanda

AND e.numeroProgressivo = p_numero

FOR UPDATE;

IF v_stato IS NULL THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Elemento inesistente';

END IF;

IF v_stato <> 'IN_ATTESA' THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Elemento gia preparato';

END IF;

SELECT ruolo INTO v_ruolo FROM Dipendente WHERE username = p_username;

```
IF v_ruolo IS NULL THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Dipendente inesistente';
```

```
END IF;
```

```
IF (v_tipo = 'PIZZA' AND v_ruolo <> 'PIZZAIOLO')
```

```
OR (v_tipo = 'BEVANDA' AND v_ruolo <> 'BARMAN') THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Il ruolo non e abilitato a preparare questo prodotto';
```

```
END IF;
```

```
UPDATE ElementoComanda
```

```
SET stato = 'PRONTO', usernamePreparatore = p_username
```

```
WHERE IDComanda = p_idComanda AND numeroProgressivo = p_numero;
```

```
COMMIT;
```

```
END;
```

G1 — sp_apri_comanda_asporto

Apri una comanda da asporto. Non richiede transazione, effettuando una sola scrittura senza precondizioni da verificare.

```
DROP PROCEDURE IF EXISTS sp_apri_comanda_asporto$$
```

```
CREATE PROCEDURE sp_apri_comanda_asporto(
```

```
    IN p_username VARCHAR(32),
```

```
    OUT p_idComanda INT UNSIGNED
```

```
)
```

```
BEGIN
  INSERT INTO Comanda (tipo, usernameDipendente)
  VALUES ('ASPORTO', p_username);
  SET p_idComanda = LAST_INSERT_ID();
END;
```

G2 — sp_apri_comanda_consegna

Registra una comanda telefonica. È transazionale perché può effettuare due scritture: l'inserimento del cliente, se non già presente in anagrafica, e quello della comanda. L'indirizzo è memorizzato sulla comanda e non sul cliente, coerentemente con quanto stabilito in progettazione concettuale.

```
DROP PROCEDURE IF EXISTS spapri_comanda_consegna$$
```

```
CREATE PROCEDURE sp_apri_comanda_consegna(
  IN p_telefono VARCHAR(20),
  IN p_nome VARCHAR(60),
  IN p_indirizzo VARCHAR(120),
  IN p_username VARCHAR(32),
  OUT p_idComanda INT UNSIGNED
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
  START TRANSACTION;

  IF NOT EXISTS (SELECT 1 FROM Cliente WHERE telefono = p_telefono) THEN
    INSERT INTO Cliente (telefono, nome) VALUES (p_telefono, p_nome);
  END IF;

  INSERT INTO Comanda (tipo, telefonoCliente, indirizzo, usernameDipendente)
  VALUES ('CONSEGNA', p_telefono, p_indirizzo, p_username);
  SET p_idComanda = LAST_INSERT_ID();
```

```
COMMIT;  
END;
```

G4 — sp_elementi_pronti_ritiro

Restituisce i prodotti pronti da ritirare dalla cucina per le comande da asporto e in consegna.

```
DROP PROCEDURE IF EXISTS sp_elementi_pronti_ritiro$$  
  
CREATE PROCEDURE sp_elementi_pronti_ritiro()  
BEGIN  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SET TRANSACTION READ ONLY;  
START TRANSACTION;  
    SELECT IDComanda, numeroProgressivo, prodotto, modalita  
    FROM v_elementi_pronti  
    WHERE modalita IN ('ASPORTO','CONSEGNA')  
    ORDER BY IDComanda;  
COMMIT;  
END;
```

G5 — sp_completa_comanda

Chiude la comanda. È la procedura transazionale più articolata, poiché realizza contemporaneamente tre regole aziendali e mantiene entrambe le ridondanze introdotte in fase di ristrutturazione.

Verifica che tutti gli elementi risultino consegnati (regola 4), calcola il totale come somma dei prezzi unitari (regola 8) e lo memorizza, e libera il tavolo se la comanda era consumata sul posto (regola 9). Le tre scritture devono essere atomiche: se il sistema si interrompesse dopo aver chiuso la comanda ma prima di liberare il tavolo, quest'ultimo resterebbe inutilizzabile.

```
DROP PROCEDURE IF EXISTS sp_completa_comanda$$  
  
CREATE PROCEDURE sp_completa_comanda(  
  
    IN p_idComanda INT UNSIGNED
```

)

BEGIN

DECLARE v_stato ENUM('APERTA','COMPLETATA') DEFAULT NULL;

DECLARE v_tavolo TINYINT UNSIGNED DEFAULT NULL;

DECLARE v_pendenti INT DEFAULT 0;

DECLARE v_totale DECIMAL(7,2) DEFAULT 0;

DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

START TRANSACTION;

SELECT stato, numeroTavolo INTO v_stato, v_tavolo

FROM Comanda WHERE ID = p_idComanda FOR UPDATE;

IF v_stato IS NULL THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Comanda inesistente';

END IF;

IF v_stato = 'COMPLETATA' THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Comanda gia completata';

END IF;

SELECT COUNT(*) INTO v_pendenti

FROM ElementoComanda

WHERE IDComanda = p_idComanda AND stato <> 'CONSEGNATO';

```
IF v_pendenti > 0 THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Esistono prodotti non ancora consegnati';

END IF;

SELECT COALESCE(SUM(prezzoUnitario), 0) INTO v_totale

FROM ElementoComanda WHERE IDComanda = p_idComanda;

UPDATE Comanda

SET stato = 'COMPLETATA', dataChiusura = NOW(), totale = v_totale

WHERE ID = p_idComanda;

IF v_tavolo IS NOT NULL THEN

    UPDATE Tavolo SET stato = 'LIBERO' WHERE numero = v_tavolo;

END IF;

COMMIT;

END;
```

G6 — sp_stampa_scontrino

Restituisce il riepilogo della comanda completata. I prezzi riportati sono quelli memorizzati sugli elementi, non quelli correnti del menu: lo scontrino resta quindi corretto anche se il gestore ha nel frattempo modificato il listino.

```
DROP PROCEDURE IF EXISTS sp_stampa_scontrino$$
```

```
CREATE PROCEDURE sp_stampa_scontrino(
```

```
    IN p_idComanda INT UNSIGNED
```

```
)
```

```
BEGIN

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SET TRANSACTION READ ONLY;

START TRANSACTION;

    SELECT c.ID, c.tipo, c.dataChiusura, c.totale,

           e.numeroProgressivo, p.nome AS prodotto, e.prezzoUnitario

    FROM Comanda c

    JOIN ElementoComanda e ON e.IDComanda = c.ID

    JOIN Prodotto p      ON e.codiceProdotto = p.codice

    WHERE c.ID = p_idComanda AND c.stato = 'COMPLETATA'

    ORDER BY e.numeroProgressivo;

COMMIT;

END;
```

G7 — sp_entrates_giornaliere

Il filtro è espresso come intervallo semiaperto sulla data di chiusura, così da consentire all'ottimizzatore di impiegare l'indice composto.

```
DROP PROCEDURE IF EXISTS sp_entrates_giornaliere$$

CREATE PROCEDURE sp_entrates_giornaliere(
    IN p_giorno DATE
)
BEGIN
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION READ ONLY;
START TRANSACTION;
```

```
SELECT COUNT(*)          AS numeroComande,
       COALESCE(SUM(totale), 0) AS incasso
FROM v_comande_completate
WHERE dataChiusura >= p_giorno
      AND dataChiusura < p_giorno + INTERVAL 1 DAY;
COMMIT;
END;
```

G8 — sp_entrare_mensili

```
DROP PROCEDURE IF EXISTS sp_entrare_mensili$$
CREATE PROCEDURE sp_entrare_mensili(
    IN p_anno INT,
    IN p_mese INT
)
BEGIN
    DECLARE v_inizio DATE;

    SET v_inizio = MAKEDATE(p_anno, 1) + INTERVAL (p_mese - 1) MONTH;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    SET TRANSACTION READ ONLY;
    START TRANSACTION;

    SELECT COUNT(*)          AS numeroComande,
           COALESCE(SUM(totale), 0) AS incasso
    FROM v_comande_completate
    WHERE dataChiusura >= v_inizio
          AND dataChiusura < v_inizio + INTERVAL 1 MONTH;

    COMMIT;
END;
```

G9 — sp_salva_prodotto

Inserisce un nuovo prodotto o aggiorna quello esistente, comprendendo anche la sospensione temporanea della disponibilità. Le due operazioni della specifica sono state unificate perché differiscono solo per i campi valorizzati, e mantenerle separate avrebbe richiesto al client di sapere in anticipo quale delle due invocare.

```
DROP PROCEDURE IF EXISTS sp_salva_prodotto$$
```

```
CREATE PROCEDURE sp_salva_prodotto(  
    IN p_codice    SMALLINT UNSIGNED,  
    IN p_nome     VARCHAR(60),  
    IN p_prezzo   DECIMAL(5,2),  
    IN p_tipo     ENUM('PIZZA','BEVANDA'),  
    IN p_disponibile BOOLEAN  
)  
BEGIN  
    IF p_codice IS NULL THEN  
        INSERT INTO Prodotto (nome, prezzo, tipo, disponibile)  
        VALUES (p_nome, p_prezzo, p_tipo, p_disponibile);  
        SELECT LAST_INSERT_ID() AS codice;  
    ELSE  
        UPDATE Prodotto  
        SET nome = p_nome, prezzo = p_prezzo,  
            tipo = p_tipo, disponibile = p_disponibile  
        WHERE codice = p_codice;  
  
        IF ROW_COUNT() = 0 THEN  
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Prodotto inesistente';  
        END IF;  
    END IF;  
END;
```

G10 — sp_registra_dipendente e sp_rimuovi_dipendente

La prima calcola il digest della password prima di memorizzarla, così che il valore in chiaro non transiti mai oltre il confine della procedura. La seconda impedisce la rimozione di un dipendente che abbia comande o preparazioni registrate a suo nome. La verifica non si limita alle comande aperte: le righe storiche conservano il riferimento a chi le ha gestite, e cancellare il dipendente comprometterebbe la tracciabilità dei dati contabili, oltre a violare i vincoli di integrità referenziale. È la stessa logica che impedisce di modificare una comanda completata.

```
DROP PROCEDURE IF EXISTS sp_registra_dipendente$$
```

```
CREATE PROCEDURE sp_registra_dipendente(  
    IN p_username VARCHAR(32),  
    IN p_password VARCHAR(64),  
    IN p_nome    VARCHAR(40),  
    IN p_cognome VARCHAR(40),  
    IN p_ruolo   ENUM('CAMERIERE','PIZZAIOLO','BARMAN','GESTORE')  
)  
BEGIN  
    INSERT INTO Dipendente (username, password, nome, cognome, ruolo)  
    VALUES (p_username, SHA2(p_password, 256), p_nome, p_cognome, p_ruolo);  
END;
```

```
CREATE PROCEDURE sp_rimuovi_dipendente(  
    IN p_username VARCHAR(32)  
)  
BEGIN  
    IF EXISTS (SELECT 1 FROM Comanda WHERE usernameDipendente = p_username)  
    OR EXISTS (SELECT 1 FROM ElementoComanda WHERE usernamePreparatore = p_username)  
    THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Il dipendente ha operazioni registrate e non puo essere rimosso';  
    END IF;  
  
    DELETE FROM Dipendente WHERE username = p_username;
```

```
IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Dipendente inesistente';
END IF;
END;
```

Procedure accessorie

Tre procedure di sola lettura non compaiono nella tavola delle operazioni perché non costituiscono operazioni autonome, ma sono indispensabili all'applicazione per raccogliere i dati necessari a invocare quelle principali: il menu corrente per comporre le comande, l'elenco dei tavoli liberi per l'assegnazione e quello delle comande aperte per selezionare su quale operare.

```
DROP PROCEDURE IF EXISTS sp_lista_menu$$
CREATE PROCEDURE sp_lista_menu()
BEGIN
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION TO READ ONLY;
START TRANSACTION;
    SELECT codice, nome, prezzo, tipo
    FROM Prodotto
    WHERE disponibile = TRUE
    ORDER BY tipo, nome;
COMMIT;
END;
```

```
DROP PROCEDURE IF EXISTS sp_tavoli_liberi$$
CREATE PROCEDURE sp_tavoli_liberi()
BEGIN
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET TRANSACTION TO READ ONLY;
START TRANSACTION;
    SELECT numero, posti
    FROM Tavolo
    WHERE stato = 'LIBERO'
```

```
ORDER BY numero;  
COMMIT;  
END;
```

```
DROP PROCEDURE IF EXISTS sp_comande_aperte$$  
CREATE PROCEDURE sp_comande_aperte()  
BEGIN  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SET TRANSACTION TO READ ONLY;  
START TRANSACTION;  
    SELECT ID, tipo, dataApertura, numeroTavolo, telefonoCliente, indirizzo  
    FROM Comanda  
    WHERE stato = 'APERTA'  
    ORDER BY dataApertura;  
COMMIT;  
END;
```